Learning Probabilistic Obstacle Spaces from Data-driven Uncertainty using Neural Networks

Jun Xiang, Jun Chen, *Senior Member, IEEE*

***Abstract*—Identifying the obstacle space is crucial for path planning. However, generating an accurate obstacle space remains a significant challenge due to various sources of uncertainty, including motion, behavior, and perception limitations. Even though an autonomous system can operate with an inaccurate obstacle space by being over-conservative and using redundant sensors, a more accurate obstacle space generator can reduce both path planning costs and hardware costs. Existing generation methods that generate high-quality output are all computationally expensive. Traditional methods, such as filtering, sensor fusion and data-driven estimators, face significant computational challenges or require large amounts of data, which limits their applicability in realistic scenarios. In this paper, we propose leveraging neural networks, commonly used in imitation learning, to mimic expert methods for modeling uncertainty and generating confidence regions for obstacle positions, which we refer to as the probabilistic obstacle space. The network is trained using a multi-label, supervised learning approach. We adopt a fine-tuned convex approximation method as the expert to construct training datasets. After training, given only a small number of samples, the neural network can accurately replicate the probabilistic obstacle space while achieving substantially faster generation speed. Moreover, the resulting obstacle space is convex, making it more convenient for subsequent path planning.**

***Index Terms*—Path planning, obstacle space, uncertainty, neural networks**

I. Introduction

AUTONOMOUS vehicles operations face various uncertainties and hazards [1]. One of the most important tasks of any autonomous vehicle is to move without collisions, in other words, to remain within a safe space at all times [2]. Safe space is also called safe region [3], obstacle-free space [4] in other works. The first step to find a safe space is usually to find the obstacle space. While many methods exist for generating a safe space under the assumption of a known obstacle space [5], this assumption is often unrealistic in practice, and only a few approaches [6], [7] extend to uncertain obstacles. Meanwhile, many previous path planning algorithms assume the obstacle space is polygonal [8], [9] due to the simplicity of collision checking and the compactness of the representation. The area of the obstacle space should be minimized, as a larger safe space typically results in smoother and shorter trajectories while also reducing the overall planning cost [4]. Building on these insights, this paper proposes a novel method to construct a polygonal *probabilistic obstacle space* that is both compact and capable of representing uncertain obstacles.

In practice, safety is ensured by declaring a collision whenever the distance between an agent and an obstacle falls below a threshold, even without physical contact [10]. In this work, we focus on obstacles with relatively small volumes, such as flights or UAVs in airspace [11], where the obstacle space is determined primarily by position rather than precise geometry, a common simplification known as the point mass model. An obstacle is certain if a deterministic value represents its position, and uncertain if a probability distribution describes its position. The probabilistic obstacle space is then defined as the confidence region of this distribution.

Uncertainty of an obstacle can be caused by many reasons. The first common reason is perception uncertainty. All of the position-locating sensors, including GPS [12], camera [13], Lidar [14], and radar [15], are subject to measurement noise. It is necessary to construct a probabilistic obstacle space from noisy measurements. Sensor Uncertainty Mitigation (SUM) methods [16] can compute bounds on the errors, but it has a very high computational cost and require diverse and representative data. Meanwhile, those over-conservative approaches make safe spaces very small, making it difficult for planning algorithms to find feasible solutions

Behavioral uncertainty can also render an obstacle uncertain, since different intents lead to entirely different trajectories [17]. Traditional data-driven methods [18], [19] can estimate intent distributions when sufficient historical data are available. However, such distributions are often highly complex and require many parameters to represent, which makes it impractical to directly translate them into obstacle spaces.

Another common source of uncertainty is motion. Even obstacles following a fixed path can exhibit positional variability [6], [20]. For instance, as shown in Fig. 2, a flight may follow its planned trajectory yet still wander around it. Consequently, even with state-of-the-art trajectory predictors, the error between the predicted and actual positions can never be eliminated. The M-estimator of Tukey [21] has been used to estimate obstacle motion from single-camera images, but it is computationally expensive and requires careful parameter tuning.

In this paper, we propose a supervised learning approach that employs multi-label training to train a transformer-based neural network for generating polygonal probabilistic obstacle spaces under uncertainty. Numerical experiments demonstrate that the network can reliably reproduce probabilistic obstacle spaces when the input samples are drawn from obstacle types it has learned. In addition, the proposed method achieves substantially faster generation times compared to baseline approaches. The method offers four key advantages:

- *Advantage 1*: The approach does not rely on dynamic models of uncertain obstacles. Instead, it can construct

The authors are with the Department of Aerospace Engineering, San Diego State University, San Diego, CA 92182 USA. (emails: jxiang9143@sdsu.edu; jun.chen@sdsu.edu).

obstacle spaces from only a few noisy measurements or partial observations, reducing the sensing and modeling requirements.
- *Advantage 2*: All generated obstacle spaces are convex and linear, which simplifies their integration into path planning algorithms and ensures computational tractability.
- *Advantage 3*: Training requires only a small amount of labeled data, which lowers the burden of data collection and annotation and makes the approach practical for large-scale deployment.
- *Advantage 4*: The generation process is extremely fast, enabling real-time use in online planning and decision-making systems.

II. Problem Statement

The main goal of this paper is to generate the *probabilistic obstacle space* of an uncertain obstacle from a *limited data sample* using a *neural network*:

$$o^{\alpha} = NN(w, \mathbf{s}), \tag{1}$$

where o^{α} is the probabilistic obstacle space at confidence level $\alpha \in (0, 1)$, w denotes the neural network parameters, and $\mathbf{s} = {s^i}_{i=1}^n$ is the set of observed n samples.

Definition (Probabilistic Obstacle Space). Let o^{true} be the true occupied region of an uncertain obstacle o, and let $\mathbb{P}$ denote the underlying probability measure that characterizes uncertainty in the obstacle's position. A set o^{α} is called a probabilistic obstacle space at confidence level α if it satisfies

$$\mathbb{P}[a \in o^{true} \mid a \notin o^{\alpha}] < 1 - \alpha. \tag{2}$$

That is, if the agent a avoids o^{α}, then the probability of entering the true obstacle region o^{true} is bounded above by $1 - \alpha$.

Since o^{true} is generally unknown due to imperfect perception and prediction, o^{α} provides a tractable confidence-region approximation. Our definition generalizes several prior notions, including the ϵ-shadow of obstacle spaces [6], [7], Gaussian-distributed obstacles [22], adaptive bounding boxes [23], and α-probabilistic occupied sets [24].

The neural network in Eq. (1) is thus trained to approximate the mapping

$$f : \mathcal{S}^n \to \mathcal{O}, \quad f(\mathbf{s}) \mapsto o^{\alpha}, \tag{3}$$

where $\mathcal{S}^n$ is the sample space of n measurements and $\mathcal{O}$ is the family of convex polygonal sets in $\mathbb{R}^d$.

We illustrate three representative examples of probabilistic obstacle spaces and their associated data samples.

Motion uncertainty: Consider the scenario of a moving object following a fixed planned trajectory. However, many factors, such as environmental disturbances, control uncertainties, and human factors, can introduce uncertainty in the actual trajectory. Therefore, as shown in Fig. 1, a single planned trajectory may correspond to infinitely many possible realizations of motion, all of which the ego agent must avoid. To capture this variability, we construct a probabilistic obstacle space that encompasses the possible deviations. Formally, we define the sample set $\mathbf{s} = {s^i}_{i=1}^n$ as the trajectories of n flights following the same planned path.

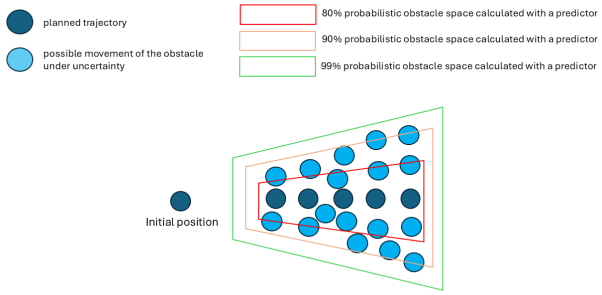


Fig. 1. Motion uncertainty: even with a planned trajectory, the actual obstacle position is affected by disturbances and uncertainties.

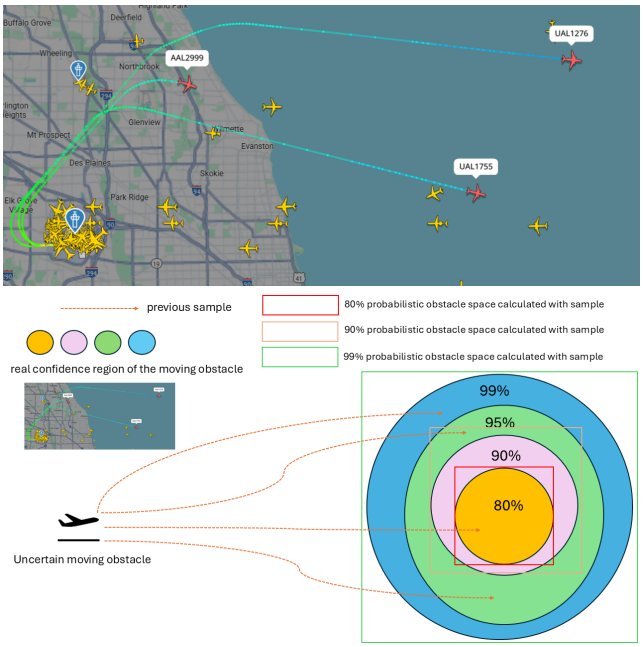


Fig. 2. Behavior uncertainty: flights taking off from the same runway with a different destination will have a different route

Behavior uncertainty: Another important source of uncertainty arises from the intent of other agents. Consider an ego UAV operating in terminal airspace near a busy airport. Although aircraft follow air traffic control clearances and standard procedures, the UAV does not know in advance which clearance each departure will receive. As shown in Fig. 2, multiple aircraft may depart from the same runway within a narrow time window, and each departure may be cleared for a different standard instrument departure depending on destination, weather, and traffic flow. From the UAV's perspective, the resulting obstacle space is therefore governed by a probability distribution over several candidate routes rather than a single deterministic path. Because such intent distributions are complex and rarely available in closed form, we approximate them by constructing the probabilistic obstacle space from $\mathbf{s}$, defined as the trajectories of n flights that

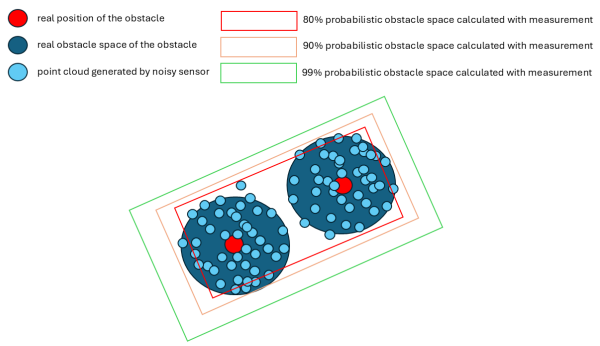


Fig. 3. Sensor uncertainty: the position of two adjacent obstacles whose positions are estimated using noisy sensor measurements is uncertain

have previously departed from the same runway.

Sensor uncertainty: Fig. 3 illustrates two adjacent circular obstacles whose positions are estimated using noisy sensor data. Because the measurements are corrupted by noise, the exact obstacle positions cannot be determined without calibration or advanced signal processing techniques. Furthermore, a single measurement outcome may be consistent with multiple possible true obstacle positions. To account for this, we define $\mathbf{s}$ as the set of n noisy sensor measurements, from which a probabilistic obstacle space is constructed.

Since the obstacle space is inherently probabilistic, its exact boundary may be irregular or complex. For path planning, however, a tractable geometric representation is preferred. In particular, polygonal approximations are widely used because they provide a convex, linear structure that can be efficiently integrated into optimization-based planners. To this end, instead of directly generating the full probabilistic obstacle space, we represent it using a convex polygon defined by its vertices. Specifically, the neural network outputs four vertices that characterize the polygonal approximation of the probabilistic obstacle space:

$$\tilde{\mathcal{V}} = NN(w, \mathbf{s}), \tag{4}$$

where w denotes the trained parameters of the neural network and $\tilde{\mathcal{V}}$ is the predicted set of polygon vertices.

III. Method

A. Supervised learning and expert method

Previous research [25] has proven that the transformer can process the visible (unmasked) parts of the data, reconstructing the entire input from the encoded, incomplete data (masked data). Therefore, if we only have a few samples from a distribution, the transformer may be able to reconstruct the entire distribution. To train the neural network, we first require expert solutions generated by an expert method to serve as supervision during the training process.

In this paper, we employ the expert method, as proposed in our previous work [26]. The expert method can generate the probabilistic obstacle space of an uncertain object. This data-driven method builds on a kernel density estimator (KDE) that can generate the probabilistic obstacle space, accelerated by a fast Fourier transform (FFT). These KDE values are then utilized within an Integer Linear Programming (ILP) framework to find a minimal parallelogram box that approximates the probabilistic obstacle space. This ILP solution effectively encapsulates the area where the state is expected to stay with high confidence.

A brief process of the ILP Heuristic Algorithm is shown in the Algorithm 1.

Algorithm 1 ILP Heuristic Algorithm

function BoxApprox($r_{\min}, c_{\min}, r_{\max}, c_{\max}, \omega_{ij}$)
  $\omega'_{ij} \leftarrow \omega_{ij}$.copy()
  $z_{ij}[1:N][1:N] \leftarrow 0$
  $r_{\min}, c_{\min} \leftarrow \arg\max_{i,j} \omega'_{ij}$
  $r_{\max} \leftarrow r_{\min}$, $c_{\max} \leftarrow c_{\min}$
  while $\sum_{i=1}^{N}\sum_{j=1}^{N} \omega_{ij} z_{ij} < \alpha \sum_i \sum_j \omega_{ij}$ **do**
    $i, j \leftarrow \arg\max_{i,j} \omega'_{ij}$
    $\omega'_{ij}[i, j] \leftarrow 0.0$
    $r_{\min} \leftarrow \min(r_{\min}, i)$, $r_{\max} \leftarrow \max(r_{\max}, i)$
    $c_{\min} \leftarrow \min(c_{\min}, j)$, $c_{\max} \leftarrow \max(c_{\max}, j)$
    $z_{ij}[r_{\min} : r_{\max}][c_{\min} : c_{\max}] \leftarrow 1$
  end while
  return z_{ij}
end function

The ILP Heuristic Algorithm approximates solutions for integer linear programming problems using heuristics. It starts with a weight matrix ω_{ij} and a zero matrix z_{ij}, setting initial indices based on ω_{ij}'s maximum values. The algorithm loops to update z_{ij} until the product of z_{ij} and ω_{ij} meets a threshold α. Each iteration zeros out the maximum ω_{ij} and adjusts boundary indices. The corresponding z_{ij} entry is set to one within these boundaries, yielding the matrix as an efficient approximation of the ILP solution.

After obtaining the expert polygon vertices $\tilde{\mathcal{V}}$, we prepare the input data for network training. Fig. 4 illustrates how different subsets of samples are presented to the neural network during training. At each training step, a small random subset of samples is selected, and the transformer is trained to generate the corresponding polygon vertices. Through this supervised learning process, the network is repeatedly exposed to multiple partial views of the same obstacle, learning to associate incomplete sample sets with the full probabilistic obstacle space. At inference time, when only a limited number of samples are available, the trained network can infer the underlying distribution of the obstacle space by leveraging the patterns it has learned from these partial-to-full mappings.

B. Multi-label learning and loss function

Recall the non-uniqueness of confidence regions [27]: for any given distribution, there exist infinitely many subsets that capture the same probability mass. Consequently, for each uncertain obstacle, there are infinitely many valid probabilistic obstacle spaces. This motivates the use of multi-label learning rather than single-label learning. Multi-label learning not only reduces computational overhead but also allows the model to share learned features across multiple labels, potentially

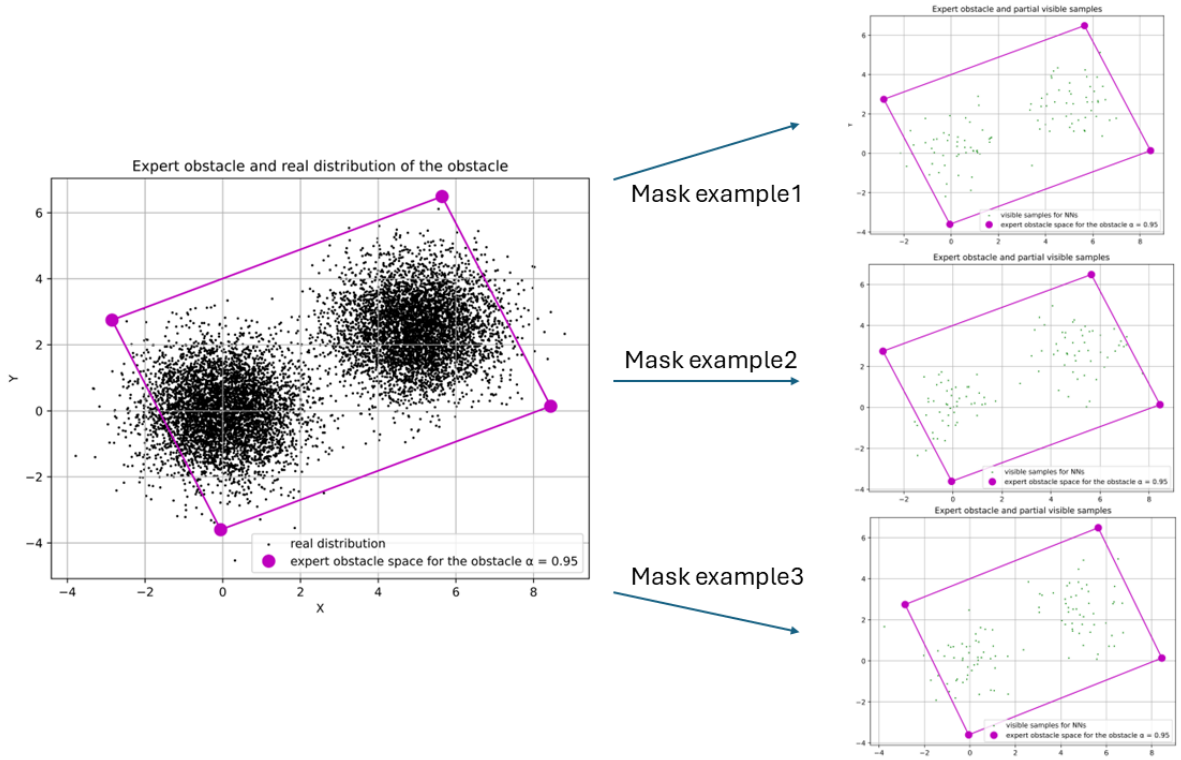


Fig. 4. Supervised learning: we iteratively pick a few samples from o^{true} as inputs to the neural network and task the network with generating the corresponding probabilistic obstacle space.

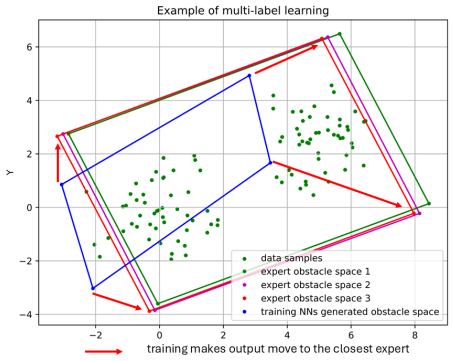


Fig. 5. Multi-label learning: during training, the network is guided to produce outputs that align with the closest expert solution.

leading to better generalization. Fig. 5 illustrates this process. Given the current network parameters, the neural network produces an output polygon. The loss is then computed between this output and each available expert probabilistic obstacle space. The smallest loss among them is used to update the network weights. In practice, multiple expert probabilistic obstacle spaces are provided for each obstacle during training. Similar to many regression-based approaches, we employ the mean squared error (MSE) loss, as shown in Eq. 5, which minimizes the average discrepancy between the network output and the expert labels.

$$\text{Loss} = \min_{j \in [1,10]} \frac{1}{8} \sum_{i=1}^{8} (\hat{y}_{ij} - y_i)^2 \quad \text{where} \quad \hat{y}_{ij} \in \tilde{\mathcal{V}} \tag{5}$$

where:

- we have 8 (four 2d vertices) elements in the output vector,
- y_i is is the generated value for the i-th element,
- $\hat{y}_{ij}$ is the j-th expert vertices for the i-th element.

C. Neural networks architecture

The overall architecture of the neural network we used is shown in Fig. 6, which is designed based on our previous work [28]. The input to the neural network consists of sampled points and the desired confidence level. Before sending these inputs to the neural network, we normalize them first. The confidence level and the sample points are then embedded in a higher dimension using a linear layer. The embedded items are concatenated together. These concatenated embeddings are then sent to the transformer layer. In the transformer, the channel representing the confidence level interacts with all the point channels. Finally, we send the confidence level channel to an output layer, resulting in four vertices of the convex polygon.

D. Input normalization, embedding, and concatenate

Because sample points are drawn from different tasks, they have very different means and variances, which can cause challenges for the training process of neural networks.

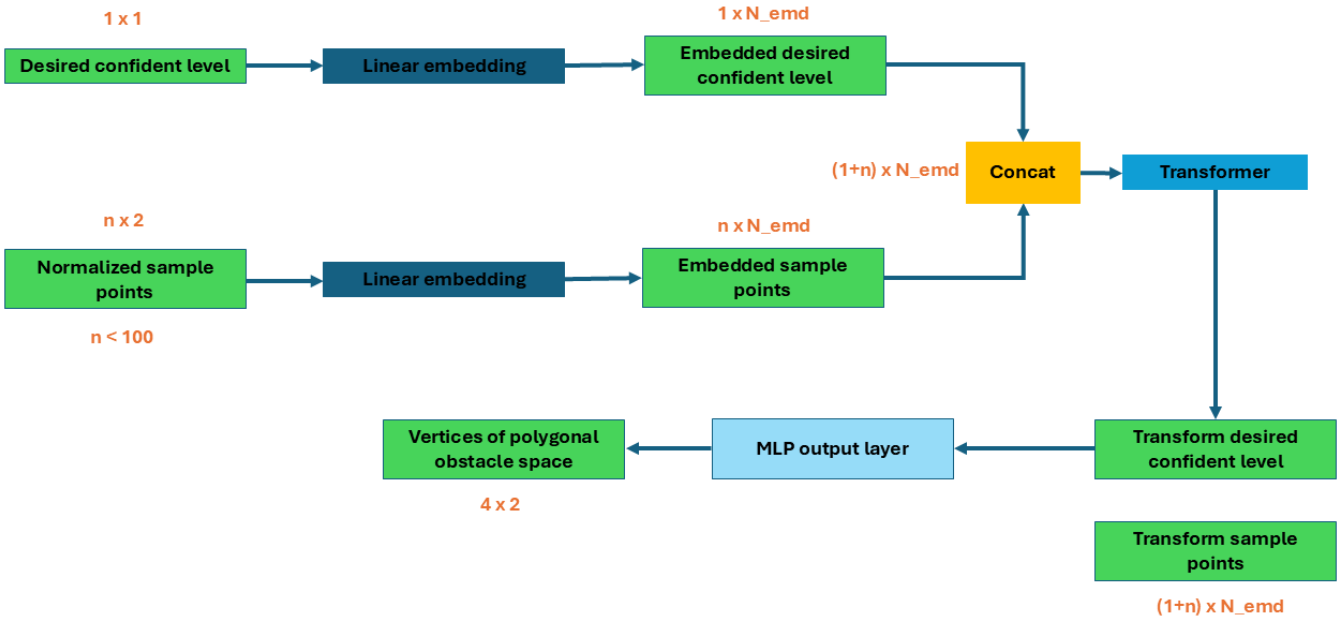


Fig. 6. Neural network architecture

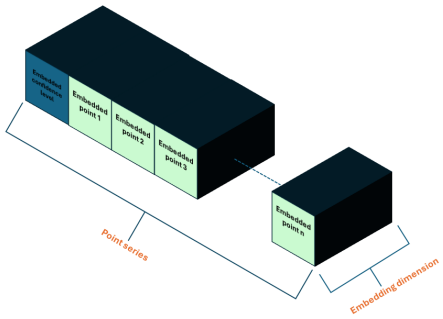


Fig. 7. Concatenated tensor

Normalizing those points before feeding them into the neural network is very crucial for several reasons. First, if some inputs have larger values and higher variance than others, they may dominate the training process. For example, if one target (or feature) has a very large value compared to others, its gradient will be disproportionately large. Second, normalization can accelerate the training process. When the scales of the inputs are not consistent, the model might spend more time adjusting weights to accommodate the varying scales of input data rather than learning meaningful patterns. Third, normalization can prevent overfitting by reducing bias toward dominant features and indirect regularization effects.

We use the Min-Max scaling [29] to normalize the sample points. The Min-Max scaling equation is defined by:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}. \tag{6}$$

Min-max scaling scales sample points to the range [0, 1] without changing the relative position between points. At the same time, since all the points are sampled from a reasonable distribution, so no outlier can heavily influence the normalization.

After normalizing, we use a single linear layer to embed the inputs. Compared to vision or natural language inputs, the inputs in our task are much more straightforward to understand. We primarily aim to determine the relationship between points rather than to interpret their meanings. Additionally, the transformer model already incorporates non-linearities. Therefore, we choose to embed the inputs into relatively low dimensions and use only a single linear layer to decrease the computing complexity.

After embedding, the point tensor has dimensions (N, C), where N is the dimension of the series of points and C is commonly referred to as the channel in the field of machine learning. We concatenated the confidence level tenor and the point tensor together along the N dimension. So, the concatenated tensor has dimensions ($N + 1$, C), while the first channel is the channel of the confidence level. The concatenated tensor then looks like the Fig. 7.

E. Transformer

For this work, we only use the transformer encoder [30] because our task requires predicting all polygon vertices at

once, instead of generating them sequentially. The encoder is composed of a stack of 6 identical layers, each layer has two sub-layers, a multi-head self-attention mechanism, and a feedforward network. We also kept the residual connection and layer normalization. Compared to other popular layers, such as the recurrent and convolutional layers, self-attention has three main advantages. First, self-attention has lower computational complexity per layer. Second, it requires fewer sequential operations since most computations can be parallelized. The third advantage is that self-attention can learn the dependencies between two points that are far from each other. For example, if we use CNNs, we may input points patch by patch, then it is difficult to learn the relationship between the first patch and the last patch. In contrast, each point interacts with all the other points at the same time in self-attention.

F. Output

In prior masked autoencoder research, mask tokens are introduced after the encoder so that the decoder can identify which missing positions need to be reconstructed. Our setting is fundamentally different. Unlike reconstruction tasks, we do not use mask tokens, since the visible samples do not carry positional information about which part of the obstacle space they come from. Moreover, sequence order is irrelevant in our problem. In natural language processing (NLP), for example, the sentences 'The dog chased the cat' and 'The cat chased the dog' convey completely different meanings despite containing the same words. In contrast, for our problem, the point sets $(1,1),(2,2)$ and $(2,2),(1,1)$ represent the same distribution. Consequently, no additional global context is required. Furthermore, the objective of our neural network is not to reconstruct the full underlying distribution but to generate a convex bounding region. Thus, a simple output layer suffices to produce the convex bound from the features extracted by the encoder.

We use a multilayered output like ViT [31] instead of a single linear layer like the original transformer. The multilayer output can further process the features extracted from the point channels and refine the final classification. We only used the transformed confidence level channel for output. The transformer allows the confidence level channel to attend to all other channels (embedded points), gathering contextual information from all the points. Finally, we need to denormalize the output, since the points we want to bound are normalized.

G. Parameters

For the transformer, the embedding dimension N_{emd} is 864. We use 6 attention heads, so each head processes 144 embedding dimensions of each channel. The number of attention layer is also 6. Optimization is performed using the Adam optimizer with a dynamic learning rate to ensure gradual convergence. Additionally, a dropout rate of 0.2 is applied to prevent overfitting by randomly omitting certain neurons during training.

IV. Result

A. Experiment setting

To evaluate the effectiveness of the proposed method, we conduct experiments on three types of uncertain obstacles: flying obstacles subject to motion uncertainty, obstacles with uncertain destinations, and obstacles whose positions are estimated using noisy sensor measurements. We feed the neural network with samples drawn from the true obstacle distribution and desired confidence level α, then evaluate whether the generated probabilistic obstacle space achieves the desired coverage, i.e., whether it bounds the specified proportion of the distribution. Obstacles of the same type share the same underlying distribution but may exhibit different geometric shapes.

1) Training: We trained a neural network that can work with varying values of α, where α can be 80%, 85%, 90%, 95% and 99%. To accelerate the training process, we prepare 100 sets of expert vertices for each desired confidence level and prepare 320,000 sets of 100 samples before training began. In each training iteration, we randomly select one set of samples and one of α as the neural network's input, then randomly assign ten sets of corresponding expert vertices as the target. We can prepare the training dataset for a known uncertain obstacle within a few hours. It supports the *Advantage 3* of the proposed method, the training dataset is easy to make.

2) Interface: We tested the trained neural networks by presenting them with 5,000 random tasks (1000 for each α) from each type of uncertain obstacle. Each task contains 100 samples randomly generated from the real distribution of the uncertain obstacle and has never been seen by the neural network. The neural network generates the corresponding o^α for those samples.

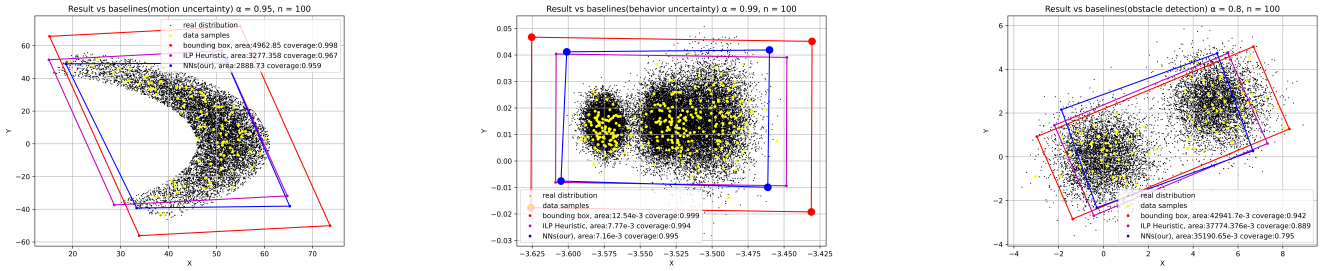
3) evaluating: To evaluate the o^α generated by the neural network, we first need to determine the coverage. We approximate the true obstacle distribution using 100,000 samples, as the underlying distributions are typically empirical. The resulting coverage was computed as the percentage of samples that fell within the generated o^α. We use mean squared error (MSE) to quantify the accuracy of the o^α:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}\left(\alpha_i - \hat{\alpha}_i\right)^2. \tag{7}$$

α_i is the real coverage of the generated o^α and $\hat{\alpha}_i$ is the desired confidence level (CL). n is 1000 because we test 1000 times for each α. We also recorded the minimum and maximum coverage values to evaluate performance stability. While a low MSE indicates that the probabilistic obstacle space is accurate, it does not guarantee correctness. Therefore, it is essential to compare the percentage of instances where α_i exceeds $\hat{\alpha}_i$. We called it the correctness rate:

$$\text{Correctness Rate} = \frac{1}{n}\sum_{i=1}^{n}\mathbf{1}[\alpha_i > \hat{\alpha}_i], \tag{8}$$

where $1[\bullet]$ is the indicator function(1 if the condition is true, 0 otherwise).



(a) motion uncertainty (b) behavior uncertainty (c) sensor uncertainty

Fig. 8. Black points are sampled from the real obstacle distribution. Colorful polygons are example outputs when the three methods take the same yellow visible points. Yellow points are enlarged for visibility.

4) baseline: We compare the proposed method with the ILP Heuristic (ILP), which is the expert method, and the bounding box (BB) method [32] [33], a popular approach to finding a polygonal approximation of the obstacle space. Both baseline methods are KDE-based. We generate o^{α} with BB and ILP for the same obstacle as the baseline. Fig. 8 are examples of obstacle distribution and o^{α} generated by different methods given the same samples. However, since BB and ILP are not learning-based methods and do not require a training process, a direct comparison may not be entirely fair. To address this, we also provide the baseline methods with 20 times more samples to enable a more thorough evaluation of our approach. In addition to these baselines, we implement the MINLP Optimal Algorithm [34] to compute the smallest convex region that satisfies the coverage requirement. All the experiences are run in Intel(R) Core(TM) i9-12900KF and NVIDIA GeForce RTX 3090.

B. Motion Uncertainty

For motion uncertainty, we evaluate flying obstacles subject to stochastic deviations in their trajectories. We simulate a flight using the Dubins vehicle model, where both speed and heading angle are sampled from a truncated Gaussian distribution. The resulting obstacle distribution for this simulated flight is shown in Fig. 8a.

From Tables I and II, our method demonstrates remarkable accuracy even with a small number of samples. For instance, at 90% confidence level (CL), NN achieves an MSE of 0.23% with a range of 89.40%–90.91%, whereas BB with 100 samples exhibits a much larger error of 7.36% and wider range (89.60%–99.99%). Even when the baselines are provided with 20× more samples, NN maintains superior accuracy and stability.

Correctness rates in Table III further highlight the robustness of our approach. NN achieves above 98% correctness at all CLs, while ILP often falls below 60% with limited samples. BB can achieve high correctness, but only at the cost of significantly larger probabilistic obstacle space areas, which reduces path planning efficiency. In contrast, NN produces obstacle spaces closer to the optimal size, especially at higher CLs.

C. Behavior Uncertainty

For behavior uncertainty, we evaluate flying obstacles with uncertain destinations. The real obstacle distribution is generated using a Gaussian Mixture Regression (GMR) trajectory predictor, which estimates the probability distribution of a flight's future positions in terminal airspace [35]. Given the past trajectory, the predictor models multiple possible future routes as a mixture of Gaussians. The resulting obstacle distribution for this predicted flight is shown in Fig. 8b.

Results under behavior uncertainty (Tables IV–VI) reveal the strength of the learning-based approach. Despite the increased distributional complexity, NN maintains low MSE (below 2% for CL $\geq$ 85%) and narrow coverage ranges. By comparison, BB consistently overshoots the target coverage, while ILP suffers from high variance and instability, particularly with limited data.

The correctness rates again demonstrate the advantage of NN. At 90% CL, NN achieves nearly perfect correctness (99.8%), whereas ILP with 300 samples drops to 69.2%. Importantly, NN generates obstacle spaces that are consistently smaller than BB while maintaining high coverage reliability.

D. Sensor Uncertainty

For sensor uncertainty, we evaluate obstacles whose positions are estimated from noisy measurements. In this scenario, the true positions of two adjacent obstacles are observed through a sensor corrupted by Gaussian noise, resulting in uncertainty about their exact locations. The real obstacle distribution is obtained by sampling noisy measurements of the true positions and constructing the corresponding probabilistic obstacle space. The resulting distribution is illustrated in Fig. 8c.

Under sensor uncertainty (Tables VII–IX), NN achieves the lowest MSE across all CLs, e.g., 0.24% at 90% CL, compared to 8.20% for BB (100 samples) and 4.98% for ILP (100 samples). Even with 20× more samples, BB and ILP fail to match NN's accuracy and stability.

Correctness rates remain above 98% for NN, ensuring that the generated obstacle spaces are reliable. Moreover, the obstacle space areas are significantly closer to the optimal solution compared with BB and ILP. This result keeps demonstrating

TABLE I
MSE AND RANGE ON MOTION UNCERTAINTY DISTRIBUTION COMPARED TO BASELINE WITH A SMALL NUMBER OF SAMPLES

	NN(our method)	BB(100)	ILP(100)
CL	MSE(range)	MSE(range)	MSE(range)
80%	0.32%(78.95% - 80.44%)	11.98%(80.28% - 99.80%)	2.83%(75.04% - 91.87%)
85%	0.25%(84.44% - 86.21%)	9.88%(85.39% - 99.91%)	2.95%(75.24% - 94.01%)
90%	0.23%(89.40% - 90.91%)	7.36%(89.60% - 99.99%)	2.69%(75.63% - 98.43%)
95%	0.49%(95.22% - 96.02%)	4.32%(92.60% - 100%)	2.20%(85.67% - 99.88%)
99%	0.98%(99.96% - 99.99%)	1.33%(98.03% - 100%)	1.12%(97.86% - 100%)

TABLE II
MSE AND RANGE ON MOTION UNCERTAINTY COMPARE TO BASELINE WITH A LARGE NUMBER OF SAMPLE

	NN(our method)	BB(2000)	ILP(2000)
CL	MSE(range)	MSE(range)	MSE(range)
80%	0.32%(78.95% - 80.44%)	12.73%(88.66% - 96.29%)	2.01%(75.36% - 84.46%)
85%	0.25%(84.44% - 86.21%)	10.98%(91.93% - 98.82%)	2.31%(81.43% - 90.18%)
90%	0.23%(89.40% - 90.91%)	8.63%(95.93% - 99.75%)	1.90%(87.40% - 94.16%)
95%	0.49%(95.22% - 96.02%)	4.81%(98.59% - 100%)	2.72%(94.35% - 99.06%)
99%	0.98%(99.96% - 99.99%)	0.99%(99.96% - 100%)	0.98%(99.19% - 100%)

TABLE III
CORRECTNESS RATE/PROBABILISTIC OBSTACLE SPACE AREA FOR EACH DESIRED CONFIDENCE LEVEL ON MOTION UNCERTAINTY

	Correctness rate					
CL	**NN**	BB(100)	BB(2000)	ILP(100)	ILP(2000)	optimal(10000)
80%	98.7%	100%	100%	57.7%	79.8%	100%
85%	99.8%	100%	100%	52.4%	64.4%	100%
90%	99.7%	99.8%	100%	52.8%	82.3%	100%
95%	100%	99.4%	100%	74.8%	98.9%	100%
99%	100%	61%	100%	59.7%	100%	100%
CL	Probabilistic obstacle space area					
80%	1415.07	2342.52	2309.52	1783.52	1514.04	1172.71
85%	1762.30	2790.63	2815.55	2138.71	1902.27	1579.52
90%	2229.01	3367.96	3470.37	2652.60	2365.27	2107.79
95%	3530.87	4525.46	4576.97	3382.33	3374.61	2742.49
99%	4833.31	5749.98	6127.58	4967.29	4820.67	3711.22

NN's ability to generalize well under noisy measurement conditions while avoiding excessive conservatism.

E. Overall Discussion

Across motion, behavior, and sensor uncertainty, the proposed NN consistently outperforms traditional BB and ILP methods. Three key observations can be drawn:

- **Accuracy with Few Samples:** NN achieves high accuracy and low MSE even with very limited data, whereas baselines require 20× more samples to approach similar performance.
- **Reliability:** NN maintains correctness rates above 93% in all scenarios, ensuring that the generated probabilistic obstacle spaces meet or exceed the desired confidence levels.
- **Efficiency:** NN produces compact obstacle spaces that are significantly smaller than BB while avoiding the instability of ILP. This efficiency is particularly critical in path planning applications where over-approximation reduces safe space.

These results validate the effectiveness of the proposed learning-based framework in generating accurate and reliable probabilistic obstacle spaces under diverse forms of uncertainty.

V. GENERATING SPEED ANALYSIS

Generating time is crucial in deciding whether a method can be used in online applications. Therefore, we compare the time to generate one correct probabilistic obstacle space by each method to prove our method is more useful in online applications. Fig. 9 illustrates the average time cost for generating effective outcomes. The results clearly demonstrate that our method significantly outperforms the baseline approaches, achieving execution times that are an order of magnitude faster. The result supports that the *Advantage 4* of the proposed method, NN can generate a useful probabilistic obstacle space very fast.

VI. CONCLUSION AND FUTURE

In this paper, we propose using supervised learning to train a transformer-based neural network to generate a confidence region of the real obstacle space distribution, which we define as the probabilistic obstacle space. Once trained, the neural network can robustly generate vertices of the polygon that cover the desired percentage of the real obstacle region, only given a few samples. The primary advantage of the proposed method is its extremely fast generation time while maintaining performance comparable to that of the baseline methods. Meanwhile, training is also very simple because training inputs

TABLE IV
MSE AND RANGE ON BEHAVIOR UNCERTAINTY COMPARE TO BASELINE WITH A SMALL NUMBER OF SAMPLE

	NN(our method)	BB(300)	ILP(300)
CL	MSE(range)	MSE(range)	MSE(range)
80%	1.79%(79.54% - 84.27%)	11.31%(84.04% - 96.41%)	2.74%(75.00% - 88.70%)
85%	1.76%(82.34% - 88.25%)	9.56%(88.09% - 98.02%)	2.62%(75.31% - 90.94%)
90%	1.13%(88.31% - 91.8%)	7.37%(93.11% - 99.40%)	1.76%(81.48% - 96.56%)
95%	0.85%(94.14$ - 97.83%)	4.09%(95.82% - 99.95%)	1.11%(90.61% - 98.80%)
99%	0.58%(99.18% - 99.94%)	0.80%(98.42% - 100%)	0.40%(97.03% - 99.95%)

TABLE V
MSE AND RANGE ON BEHAVIOR UNCERTAINTY COMPARE TO BASELINE WITH A LARGE NUMBER OF SAMPLE

	NN(our method)	BB(6000)	ILP(6000)
CL	MSE(range)	MSE(range)	MSE(range)
80%	1.79%(79.54% - 84.27%)	10.47%(85.60% - 93.33%)	2.40%(75.00% - 81.90%)
85%	1.76%(82.34% - 88.25%)	9.45%(90.69% - 96.35%)	3.01%(85.60% - 93.33%)
90%	1.13%(88.31% - 91.8%)	7.44%(95.89% - 98.67%)	1.37%(85.62% - 94.55%)
95%	0.85%(94.14$ - 97.83%)	4.22%(97.98% - 99.67%)	0.74%(92.35% - 97.50%)
99%	0.58%(99.18% - 99.94%)	0.91%(99.72% - 99.97%)	0.25%(98.28% - 99.61%)

TABLE VI
CORRECTNESS RATE/PROBABILISTIC OBSTACLE SPACE AREA FOR EACH DESIRED CONFIDENCE LEVEL ON BEHAVIOR UNCERTAINTY

	Correctness rate					
CL	**NN**	BB(300)	BB(6000)	ILP(300)	ILP(6000)	optimal(30000)
80%	93.1%	100%	100%	10.8%	3.9%	100%
85%	95.2%	100%	100%	27.9%	16.2%	100%
90%	99.8%	100%	100%	69.2%	55.8%	100%
95%	95.6%	100%	100%	68.2%	65.4%	100%
99%	100%	99.7%	100%	56.5%	75.8%	100%
CL	Probabilistic obstacle space area					
80%	2.75E-3	3.64E-3	3.46E-3	2.59E-3	2.39E-3	2.15E-3
85%	3.44E-3	4.41E-3	4.35E-3	2.99E3	2.89E-3	2.39E-3
90%	4.41E-3	7.37E-3	5.81E-3	3.83E-3	3.67E-3	2.99E-3
95%	5.69E-3	7.89E-3	7.98E-3	5.03E-3	4.87E-3	3.95E-3
99%	9.21E-3	11.19E-3	11.16E-3	8.01E-3	7.63E-3	6.46E-3

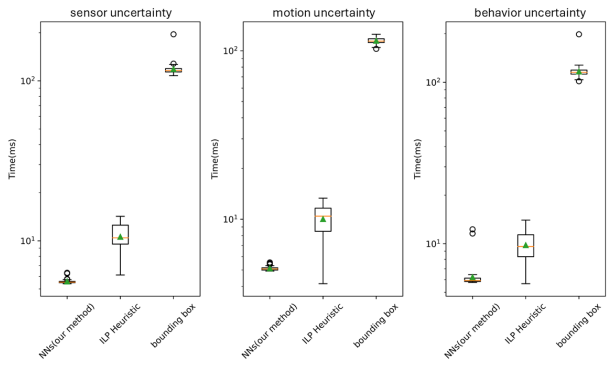


Fig. 9. Generating speed comparison

can be generated very fast. Only a few experts are required for the labels. No dynamic information about the obstacle such as disturbance is required.

ACKNOWLEDGMENT

The authors would like to acknowledge the support from the National Science Foundation under Grants CCF-2402689. Any opinions, findings, conclusions, or recommendations expressed in this paper are those of the authors and do not reflect the views of NSF.

REFERENCES

[1] E. L. Thompson, A. G. Taye, W. Guo, P. Wei, M. Quinones, I. Ahmed, G. Biswas, J. Quattrociocchi, S. Carr, U. Topcu *et al.*, "A survey of evtol aircraft and aam operation hazards," in *AIAA AVIATION 2022 Forum*, 2022, p. 3539.

[2] C. Dawson, A. Jasour, A. Hofmann, and B. Williams, "Provably safe trajectory optimization in the presence of uncertain convex obstacles," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 6237–6244.

[3] F. Gao and S. Shen, "Online quadrotor trajectory generation and autonomous navigation on point clouds," in *2016 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, 2016, pp. 139–146.

[4] Q. Wang, Z. Wang, M. Wang, J. Ji, Z. Han, T. Wu, R. Jin, Y. Gao, C. Xu, and F. Gao, "Fast iterative region inflation for computing large 2-d/3-d convex regions of obstacle-free space," *IEEE Transactions on Robotics*, 2025.

[5] V. Savic, H. Wymeersch, and E. G. Larsson, "Target tracking in confined environments with uncertain sensor positions," *IEEE Transactions on Vehicular Technology*, vol. 65, no. 2, pp. 870–882, 2015.

[6] B. Axelrod, L. P. Kaelbling, and T. Lozano-Pérez, "Provably safe robot navigation with obstacle uncertainty," *The International Journal of Robotics Research*, vol. 37, no. 13-14, pp. 1760–1774, 2018.

[7] L. Shimanuki and B. Axelrod, "Hardness of motion planning with obstacle uncertainty in two dimensions," *The International Journal of Robotics Research*, vol. 40, no. 10-11, pp. 1151–1166, 2021.

TABLE VII
MSE AND RANGE ON SENSOR UNCERTAINTY DISTRIBUTION COMPARE TO BASELINE WITH A SMALL NUMBER OF SAMPLE

	NN(our method)	BB(100)	ILP(100)
CL	MSE(range)	MSE(range)	MSE(range)
80%	0.42%(79.12% - 82.05%)	11.04%(87.56% - 99.42%)	3.66%(72.00% - 93.54%)
85%	0.35%(84.45% - 86.63%)	10.18%(91.98% - 99.80%)	3.70%(72.17% - 95.76%)
90%	0.24%(89.51% - 91.01%)	8.20%(93.26% - 99.94%)	4.98%(86.16% - 98.78%)
95%	0.62%(94.08% - 96.29%)	4.20%(94.84% - 99.99%)	3.25%(92.45% - 99.73%)
99%	0.87%(97.66% - 99.94%)	1.87%(97.09% - 99.99%)	0.80%(98.97% - 99.99%)

TABLE VIII
MSE AND RANGE ON SENSOR UNCERTAINTY DISTRIBUTION COMPARE TO BASELINE WITH A LARGE NUMBER OF SAMPLE

	NN(our method)	BB(2000)	ILP(2000)
CL	MSE(range)	MSE(range)	MSE(range)
80%	0.42%(79.12% - 82.05%)	16.26%(93.24% - 98.43%)	4.12%(72.01% - 86.43%)
85%	0.35%(84.45% - 86.63%)	12.10%(95.65% - 98.49%)	3.15%(74.27% - 93.60%)
90%	0.24%(89.51% - 91.01%)	8.12%(96.52% - 99.63%)	4.37%(89.70% - 98.32%)
95%	0.62%(94.08% - 96.29%)	4.28%(98.64% - 99.86%)	3.22%(95.48% - 99.53%)
99%	0.87%(97.66% - 99.94%)	0.93%(99.73% - 99.99%)	0.86%(99.49% - 99.98%)

TABLE IX
CORRECTNESS RATE/PROBABILISTIC OBSTACLE SPACE AREA FOR EACH DESIRED CONFIDENCE LEVEL ON SENSOR UNCERTAINTY

	Correctness rate					
CL	**NN**	BB(100)	BB(2000)	ILP(100)	ILP(2000)	optimal(10000)
80%	99.7%	100%	100%	31.4%	14.7%	100%
85%	99.8%	100%	100%	67.3%	57.9%	100%
90%	99.8%	100%	100%	97.2%	99.8%	100%
95%	98.9%	99.9%	100%	99.1%	100%	100%
99%	98.7%	95.2	100%	99.9%	100%	100%
CL	Probabilistic obstacle space area					
80%	35.22	45.65	46.14	36.06	33.95	33.01
85%	41.58	52.16	50.12	42.42	40.69	34.23
90%	51.38	63.18	59.10	53.86	52.35	42.79
95%	64.53	75.85	71.90	67.46	66.08	55.01
99%	99.03	101.49	106.67	95.28	96.10	73.96

[8] R. Deits and R. Tedrake, "Computing large convex regions of obstacle-free space through semidefinite programming," in *Algorithmic Foundations of Robotics XI: Selected Contributions of the Eleventh International Workshop on the Algorithmic Foundations of Robotics*. Springer, 2015, pp. 109–124.

[9] T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake, "Motion planning around obstacles with convex optimization," *Science robotics*, vol. 8, no. 84, p. eadf7843, 2023.

[10] L. Wang, A. D. Ames, and M. Egerstedt, "Safety barrier certificates for collisions-free multirobot systems," *IEEE Transactions on Robotics*, vol. 33, no. 3, pp. 661–674, 2017.

[11] M. Mitici and H. A. P. Blom, "Mathematical models for air traffic conflict and collision probability estimation," *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 3, pp. 1052–1068, 2019.

[12] M. G. Ferguson, "Global positioning system (gps) error source prediction," Tech. Rep., 2000.

[13] C. Zhu, M. Joerger, and M. Meurer, "Quantifying feature association error in camera-based positioning," in *2020 IEEE/ION Position, Location and Navigation Symposium (PLANS)*. IEEE, 2020, pp. 967–972.

[14] A. Hassani and M. Joerger, "A new point-cloud-based lidar/imu localization method with uncertainty evaluation," in *Proceedings of the 34th international technical meeting of the satellite division of the institute of navigation (ion gnss+ 2021)*, 2021, pp. 636–651.

[15] Z. Hong, Y. Petillot, K. Zhang, S. Xu, and S. Wang, "Large-scale radar localization using online public maps," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 3990–3996.

[16] M. Abramson, M. Refai, M. G. Wu, and S. Lee, "Applying sensor uncertainty mitigation schemes to detect-and-avoid systems," in *AIAA Aviation 2020 Forum*, 2020, p. 3264.

[17] J. Li, X. Shi, F. Chen, J. Stroud, Z. Zhang, T. Lan, J. Mao, J. Kang, K. S. Refaat, W. Yang, E. Ie, and C. Li, "Pedestrian crossing action recognition and trajectory prediction with 3d human keypoints," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 1463–1470.

[18] J. Xiang and J. Chen, "Data-driven probabilistic trajectory learning with high temporal resolution in terminal airspace," *Journal of Aerospace Information Systems*, pp. 1–11, 2025.

[19] W. Zhi, L. Ott, and F. Ramos, "Probabilistic trajectory prediction with structural constraints," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 9849–9856.

[20] J. Miura and Y. Shirai, "Modeling motion uncertainty of moving obstacles for robot motion planning," in *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No. 00CH37065)*, vol. 3. IEEE, 2000, pp. 2258–2263.

[21] C. Demonceaux, A. Potelle, and D. Kachi-Akkouche, "Obstacle detection in a road scene based on motion analysis," *IEEE Transactions on Vehicular Technology*, vol. 53, no. 6, pp. 1649–1656, 2004.

[22] P. Wu, L. Li, J. Xie, and J. Chen, "Probabilistically guaranteed path planning for safe urban air mobility using chance constrained rrt," in *AIAA Aviation 2020 Forum*, 2020, p. 2914.

[23] A. Timans, C.-N. Straehle, K. Sakmann, and E. Nalisnick, "Adaptive bounding box uncertainties via two-step conformal prediction," in *European Conference on Computer Vision*. Springer, 2024, pp. 363–398.

[24] A. P. Vinod and M. M. K. Oishi, "Probabilistic occupancy via forward stochastic reachability for markov jump affine systems," *IEEE Transactions on Automatic Control*, vol. 66, no. 7, pp. 3068–3083, 2021.

[25] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 16 000–16 009.

[26] P. Wu and J. Chen, "Efficient box approximation for data-driven probabilistic geofencing," *Unmanned Systems*, pp. 1–12, 2023.

[27] J. Neyman, "Outline of a theory of statistical estimation based on the classical theory of probability," *Philosophical Transactions of the Royal*

Society of London. Series A, Mathematical and Physical Sciences, vol. 236, no. 767, pp. 333–380, 1937.
[28] J. Xiang and J. Chen, "Imitation learning-based convex approximations of probabilistic reachable sets," in *AIAA AVIATION FORUM AND ASCEND 2024*, 2024, p. 4356.
[29] T. Hastie, R. Tibshirani, J. H. Friedman, and J. H. Friedman, *The elements of statistical learning: data mining, inference, and prediction*. Springer, 2009, vol. 2.
[30] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
[31] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.
[32] W. Zhao and R. Wen, "The algorithm of fast collision detection based on hybrid bounding box," in *2012 International Conference on Computer Science and Electronics Engineering*, vol. 3. IEEE, 2012, pp. 547–551.
[33] C. Ericson, *Real-time collision detection*. Crc Press, 2004.
[34] P. Wu and J. Chen, "Data-driven zonotopic approximation for n-dimensional probabilistic geofencing," *Reliability Engineering & System Safety*, vol. 244, p. 109923, 2024.
[35] J. Xiang and J. Chen, "Data-driven probabilistic trajectory learning with high temporal resolution in terminal airspace," *arXiv preprint arXiv:2409.17359*, 2024.

VII. Biography Section

Jun Xiang is currently pursuing the joint Ph.D. degree at the Department of Mechanical and Aerospace Engineering, University of California at San Diego, and the Department of Aerospace Engineering, San Diego State University. He received his M.S. degree in Automotive Engineering from Clemson University and his B.S. degree in Mechanical Engineering from North Carolina State University. His research interests include motion planning and prediction of autonomous air/ground vehicle systems.

Jun Chen is an Associate Professor of Aerospace Engineering at San Diego State University. He received his M.S. and Ph.D. degrees in Aerospace Engineering from Purdue University. His research interests include control and optimization for large-scale networked dynamical systems, with applications in mechanical and aerospace engineering such as air traffic control, traffic flow management, and autonomous air/ground vehicle systems.